

Лабораторна робота №2

Синхронізація паралельних операцій за допомогою механізму подій та механізму майбутніх результатів

Мета роботи: Навчитись синхронізувати паралельні операції за допомогою механізму подій та механізму майбутніх результатів.

1.1 Теоретичні відомості

Одним із способів синхронізації паралельних операцій є проста перевірка спільного для потоків прапорця, який захищений м'ютексом, вважаючи, що один із потоків підніме цей прапорець, коли завершить свою операцію. Такий підхід має ряд недоліків, оскільки, передбачає витрачання ресурсів системи на постійну перевірку значення прапорця. Одним із способів подолання цього недоліку є застосування функції `this_thread::sleep_for()`, яка змушує потік «спати» між перевітками значення прапорця. Наприклад, за допомогою наступної конструкції:

```
unique_lock<mutex> lk(m);
while (!flag)
{
    lk.unlock();
    this_thread::sleep_for(chrono::milliseconds(100));
    lk.lock();
}
```

Наведений спосіб не позбавлений більшості недоліків, пов'язаних з опитуванням значень прапорів, тому рекомендованим способом синхронізації потоків є використання засобів стандартної бібліотеки C++, які дозволяють потоку очікувати на події. Найпростіший механізм очікування на події, які виникають у іншому потоці (наприклад, поява нового завдання), дають умовні змінні. Концептуально, умовна змінна асоційована з якоюсь подією або іншою умовою, причому, один або кілька потоків можуть чекати, коли ця умова почне виконуватись. Якщо деякий потік вирішить, що умова виконана, він може сповістити про це один або кілька потоків, які очікують умовну змінну, в результаті чого, вони відновлять роботу.

Стандартна бібліотека C++ надає не одну, а дві реалізації умовних змінних: `std::condition_variable` і `std::condition_variable_any`. Обидва класи оголошені в заголовку `<condition_variable>`. В обох випадках для забезпечення синхронізації необхідна взаємодія з м'ютексами; перший клас може працювати тільки з `std::mutex`, другий – з будь-яким класом, який відповідає мінімальним вимогам до «м'ютексоподібності». Оскільки, клас `std::condition_variable_any` є більш загальним, то його використання може обійтися дорожче з точки зору обсягу споживаної пам'яті, продуктивності та ресурсів операційної системи. Тому, якщо додаткова гнучкість не потрібна, то краще обмежитися класом `std::condition_variable`.

Для пробудження потоку, інший потік викликає функцію-член `notify_one()` об'єкта `std::condition_variable`, щоб сповістити потік (якщо такий існує), який очікує, про настання певної події. Якщо таких потоків декілька, то функція пересилає повідомлення одному з них (вибраному випадковим чином). Для повідомлення усіх потоків використовується функція-член `notify_all()`.

Потік, який очікує, викликає функцію-член `wait()` об'єкта `std::condition_variable`, передаючи їй об'єкт-блокування (м'ютекс) і лямбда-функцію, яка виражає очікувану умову. Лямбда-функції дозволяють записати анонімну функцію як частину виразу та ідеально підходять для задання предикатів для таких стандартних бібліотечних функцій, як `wait()`. Наприклад, проста лямбда-функція `[] {return !data_queue.empty(); }` перевіряє, чи черга `data_queue` не є порожня (викликаючи її метод `empty()`), тобто, що в ній є дані для обробки.

Далі функція `wait()` перевіряє умову (викликаючи передану лямбда-функцію) і повертає управління, якщо вона виконана (тобто лямбда-функція повернула `true`). Якщо умова не виконана (лямбда-функція повернула `false`), то `wait()` звільняє м'ютекс і переводить потік в стан очікування. Коли умовна змінна отримає повідомлення, відправлене потоком підготовки даних за допомогою `notify_one()`, потік обробки пробудиться, знову захопить м'ютекс і ще раз перевірить умову. Якщо умова виконана, то `wait()` поверне управління, причому м'ютекс в цей момент буде захоплений. Якщо ж умова не виконана, то потік

знову звільняє м'ютекс і відновлює очікування. Саме тому необхідно застосовувати `std::unique_lock`, а не `std::lock_guard` – потік, який очікує, повинен звільняти м'ютекс, коли знаходиться в стані очікування, і захоплювати його на виході з цього стану, а `std::lock_guard` такою гнучкістю не володіє.

У класі `std::thread` не передбачено механізму передачі в основний потік (або інше місце виклику) результатів виконання функції-потіку. Для цього застосовується механізм майбутніх результатів. У стандартній бібліотеці C++ є два різновиди майбутніх результатів, які реалізовані у формі двох шаблонів класів, які оголошені в заголовку `<future>`: унікальні майбутні результати (`std::future<>`) і розділювані майбутні результати (`std::shared_future<>`).

Одним із засобів повернення результатів з потоку є шаблон функції `std::async` (оголошений у заголовку `<future>`). Функція `std::async` дозволяє запустити асинхронне завдання, у результаті якого у поточний момент немає необхідності. Замість об'єкта `std::thread` вона повертає об'єкт `std::future`, який буде містити повернене значення, коли воно стане доступним. Коли програмі знадобиться значення, вона викликає функцію-член `get()` об'єкта-майбутнього, і тоді потік буде призупинено до готовності майбутнього результату, після чого він поверне значення.

Наприклад, для деякої функції з двома аргументами запуск `std::async` і отримання результатів матиме вигляд:

```
future<int> f1 = async(launch::async, Func, arg1, arg2);  
int res = f1.get();
```

Поведінку `std::async` можна задати у додатковому параметрі перед функцією, яка викликається. Цей параметр має тип `std::launch` і може набувати таких значень: `std::launch::deferred` – відкласти виклик функції до того моменту, коли буде викликана функція-член `wait()` або `get()` об'єкта-майбутнього; `std::launch::async` – запускати функцію в окремому потоці. У разі відкладеного виклику функція може взагалі ніколи не виконатися.

Для того, щоб передати декілька задач у один потік або розподілити задачі по конкретно заданих потоках використовується шаблон класу `std::packaged_task<>`, який пов'язує майбутній результат з функцією або об'єктом, що допускає виклик. При виклику об'єкта `std::packaged_task<>`

асоційована функція викликається і робить майбутній результат готовим, зберігаючи повернене значення у вигляді асоційованих даних. Параметром шаблону класу `std::packaged_task<>` є сигнатура функції, наприклад

```
packaged_task<void()>
```

для функції, яка не приймає ніяких параметрів і не повертає значення.

Об'єкт `std::packaged_task` допускає виклик, тому цілком вірним є наступний запис:

```
void Func(double arg1, int arg2) {...}
packaged_task<void(double, int)> task(Func);
.....
task(13.5, 258);
```

Тип значення, яке повертається, визначає тип об'єкта `std::future<>`, що повертається функцією-членом `get_future()`:

```
bool DoSomething(const string& str)
{
    .....
}

future<bool> returnSomething(const string& str)
{
    std::packaged_task<bool(const std::string&)> task(DoSomething);
    auto future = task.get_future();
    std::thread t1(std::move(task), str);
    t1.detach();
    return std::move(future);
}
```

Об'єкти `std::packaged_task<>` здебільшого використовують для наповнення деякої структури даних, яка містить завдання для заданого потоку (потоків). Наприклад:

```
queue<packaged_task<void()>> tasks;
void Packer()
{
    packaged_task<void()> task(Func);
    .....
    tasks.push(move(task));
    .....
}
void Worker()
{
    packaged_task<void()> task;
    .....
    task = move(tasks.front());
    tasks.pop();
    .....
    task();
}
```

Найбільш повний контроль за розподіленням операцій між потоками забезпечує пара `std::promise/std::future`. Шаблон `std::promise<T>` дає можливість задати значення (типу `T`), яке згодом можна буде прочитати за допомогою асоційованого об'єкта `std::future<T>`. Потік, який очікує даних, призупиняється в очікуванні майбутнього результату, тоді як потік, який постачає дані, може за допомогою `promise` встановити асоційоване значення і зробити майбутній результат готовим.

Щоб отримати об'єкт `std::future`, асоційований із заданим `std::promise`, необхідно викликати функцію-член `get_future()` аналогічно до `std::packaged_task`. Після установки значення `std::promise` (за допомогою функції-члена `set_value()`) майбутній результат стає готовим, і його можна використовувати для отримання встановленого значення.

Крім цього, `set_value()` дозволяє реалізувати вивід (у вигляді майбутнього результату) результатів обчислень (довільного типу), які здійснені у межах функцій (потоків), які повертають тип `void`:

```
void Func(int arg, promise<int> prom)
{
    prom.set_value(arg * arg);
}

int main()
{
    .....
    promise<int> prom;
    future<int> f = prom.get_future();
    thread t(Func, arg, move(prom));
    .....
    cout << "results are equal to " << f.get() << endl;
    .....
}
```

Ця властивість, також, дозволяє повертати з одного потоку-функції довільну кількість результатів без застосування глобальних структур даних.

Здебільшого, пару `std::promise/std::future` застосовують у випадках, коли результат виконаних обчислень необхідно повернути у місце очікування лише при виконанні деяких умов або у певний момент часу, оскільки, `set_value()` помічає майбутній результат як готовий до повернення у місце виклику і при цьому забезпечує «ручне» керування цим процесом.

1.2 Хід роботи

1.2.1 Створити програму, у якій запускаються два потоки `DataPreparation` (від'єднується від головного потоку) та `DataProcessing` (головний потік очікує на його завершення). У потоці `DataPreparation` відбувається зчитування цілих чисел від користувача (через стандартний пристрій вводу), які поміщаються у глобальну чергу. У цей час потік `DataProcessing` очікує на завершення введення даних. Після завершення введення користувачем даних (передбачити можливість уведення довільного числа даних) потік `DataPreparation` встановлює прапорець для запуску потоку `DataProcessing`, який перевіряє усі числа у черзі на простоту та виводить прості.

Для синхронізації потоків використати глобальний прапорець, об'єкт `std::unique_lock` та `this_thread::sleep_for(chrono::milliseconds(100))`.

1.2.2 Створити програму, яка запускає три потоки на основі функції `Waits()`, яка очікує на сигнал від потоку `Awake()` і перед початком очікування виводить повідомлення про те, що вона входить у стан очікування на стандартний пристрій вводу. Після отримання сигналу, виводиться повідомлення про те, що очікування завершено. У якості умови для функції `wait()` використати лямбда-функцію, яка перевіряє рівність значення глобальної змінної і одиниці (змінну і проініціалізувати нулем). Потік `Awake` очікує (спить) 100 мілісекунд, після чого повідомляє про те, що починає повідомляти решту (через стандартний пристрій вводу) і робить це за допомогою `notify_all()`, після чого переходить у «сон» на 100 мілісекунд. Далі (і лише зараз) він присвоює змінній і значення 1, видає повідомлення на стандартний пристрій вводу про повторне повідомлення і повторно викликає `notify_all()`.

Головний потік повинен дочекатись виконання потоку `Awake`.

Запустіть програму декілька разів, запишіть та проаналізуйте результати.

1.2.3 Створіть програму, яка створює три потоки `Thread1`, `Thread2` та `Thread3`, кожен з яких очікує на сигнал аналогічним до попереднього

завдання чином, і після сигналу виводить на стандартний пристрій вводу повідомлення: «Повідомлення з потоку...» із вказанням відповідного номеру (1, 2 або 3). Також, створюється потік Notify, який після очікування на протязі 100 мілісекунд пробуджує один з потоків за допомогою функції `notify_one()` і присвоює змінній `i` значення 1.

Головний потік повинен дочекатись виконання потоку Notify.

Запустіть програму декілька разів, запишіть та проаналізуйте результати.

1.2.4 Змініть код програми з пункту 1.2.1, щоб замінити синхронізацію на основі прапора синхронізацією на основі умовних змінних. У якості умови для функції `wait()` використати лямбда-функцію, яка перевіряє чергу на пустоту.

1.2.5 Створити програму, яка пропонує користувачеві дізнатися n -те просте число (n вводиться користувачем). Після цього, запустити на виконання обчислення цього числа за допомогою `std::async`. Далі, запропонувати користувачеві (поки йде обчислення) дізнатися, чому дорівнює значення однієї з функцій (корінь квадратний, синус, натуральний логарифм) від уведеного ним номера простого числа. Після цього, програма має вивести результат обчислень простого числа за заданим номером.

У першому випадку створити об'єкт `std::async` з параметром `std::launch::deferred`, у другому – `std::launch::async`. Порівняти час очікування на результат обчислення (наприклад, мільйонного простого числа) у обох випадках. Зробити висновки про причини різниці у затримці.

1.2.6 Створити програму, яка у головному потоці пропонує користувачеві дізнатися довільну кількість n -тих простих чисел (n вводиться користувачем), тобто опитує користувача, доки йому не набридне не отримає стоп-символ. Створити два дека (структура даних), перший з яких містить об'єкти `std::packaged_task<>`, які відповідають завданням обчислення n -го простого числа, а другий – власне номери n . Створити

окремий **незалежний** від головного потік, який виконує вказані завдання, що постачаються йому через деки.

Забезпечити безпеку доступу до деків. Забезпечити очікування головним потоком завершення усіх обчислювальних завдань та коректне завершення роботи незалежного потоку. Вивід результатів обчислень здійснювати на стандартний пристрій вводу.

- 1.2.7 Створити програму, яка пропонує користувачеві дізнатися n -те просте число (n вводиться користувачем). Створити два незалежні від головного потоки. Перший з них повертає головному потокові для виводу на стандартний пристрій вводу n -те просте число за допомогою `std::promise/std::future`, встановлює булевий `std::promise` у значення `true`, чим сигналізує другому потокові, про початок його роботи, і повертає головному потокові для виводу на стандартний пристрій вводу $(n*10)$ -те просте число за допомогою `std::promise/std::future`. Другий потік очікує на майбутній результат булевого типу (від першого потоку) і після його отримання, чекає дві секунди, після чого виводить на стандартний пристрій виводу значення квадратного кореня від числа n .

Усі потоки, крім головного, повинні повертати тип `void`. У процесі написання програми не використовувати жодних глобальних змінних чи структур даних.

Проаналізувати роботу одержаної програми та ефективність застосування пари `std::promise/std::future`.

- 1.2.8 Запишіть лістинги розроблених програм у звіт. Зробіть висновки щодо застосування механізму умовних змінних для синхронізації паралельних операцій. Зробіть висновки щодо роботи шаблонів стандартної бібліотеки на основі об'єктів майбутніх подій. Порівняйте зручність та ефективність застосування вивчених механізмів синхронізації.

1.3 Декларація про використання генеративного штучного інтелекту

Зразок: Під час виконання роботи було використано інструмент [Назва інструменту, наприклад: ChatGPT 4o / GitHub Copilot] для [вказати мету, наприклад: відлагодження помилки підключення до БД / оптимізації стилю коду / тестування самостійно написаного застосунку]. Увесь поданий результат реалізовано та оформлено здобувачем власноруч [за потреби можна уточнити верифікацію за джерелами/документацією]. Здобувач бере на себе повну відповідальність за коректність результатів та зміст лабораторної роботи.

1.4 Зміст звіту

1. Титульна сторінка та тема роботи.
2. Мета роботи.
3. Короткі теоретичні відомості (обсягом до 1 сторінки).
4. Хід роботи (результати виконаної роботи).
5. Декларація про використання генеративного штучного інтелекту.
6. Висновки.

1.5 Контрольні запитання

- 1.5.1 Як забезпечується розпаралелювання задач у операційних системах?
- 1.5.2 Яким чином забезпечене управління потоками у мові C++?
- 1.5.3 Що таке м'ютекс та які основні принципи його роботи?
- 1.5.4 Яким чином створюється м'ютекс засобами мови C++?
- 1.5.5 Що таке умовна змінна?
- 1.5.6 Яким чином створюються умовні змінні засобами мови C++?
- 1.5.7 Як змусити потік очікувати на зміну значення умовної змінної?
- 1.5.8 Що таке об'єкт майбутньої події?
- 1.5.9 Як запустити асинхронне паралельне обчислення, результат якого зберігатиметься у об'єкті майбутньої події?
- 1.5.10 Які режими виконання обчислень допускає функція `std::async`?
- 1.5.11 Чи можна здійснювати безпосередній виклик об'єкта `packaged_task`?
- 1.5.12 Як отримати об'єкт майбутнього результату, який пов'язаний із заданим об'єктом `std::promise`?